22/09/2025

Real-Time Data Pipeline Architecture

Components:

1. **Producer** → Python script simulating IoT sensor data.
2. **Amazon Kinesis Data Stream** → 2 shards for parallel ingestion.
3. **Consumers**:
    - **Lambda Function** → Filters for "ERROR" and logs to **CloudWatch Logs**.
    - **Kinesis Data Firehose** → Buffers for 60s, then delivers to **Amazon S3**.

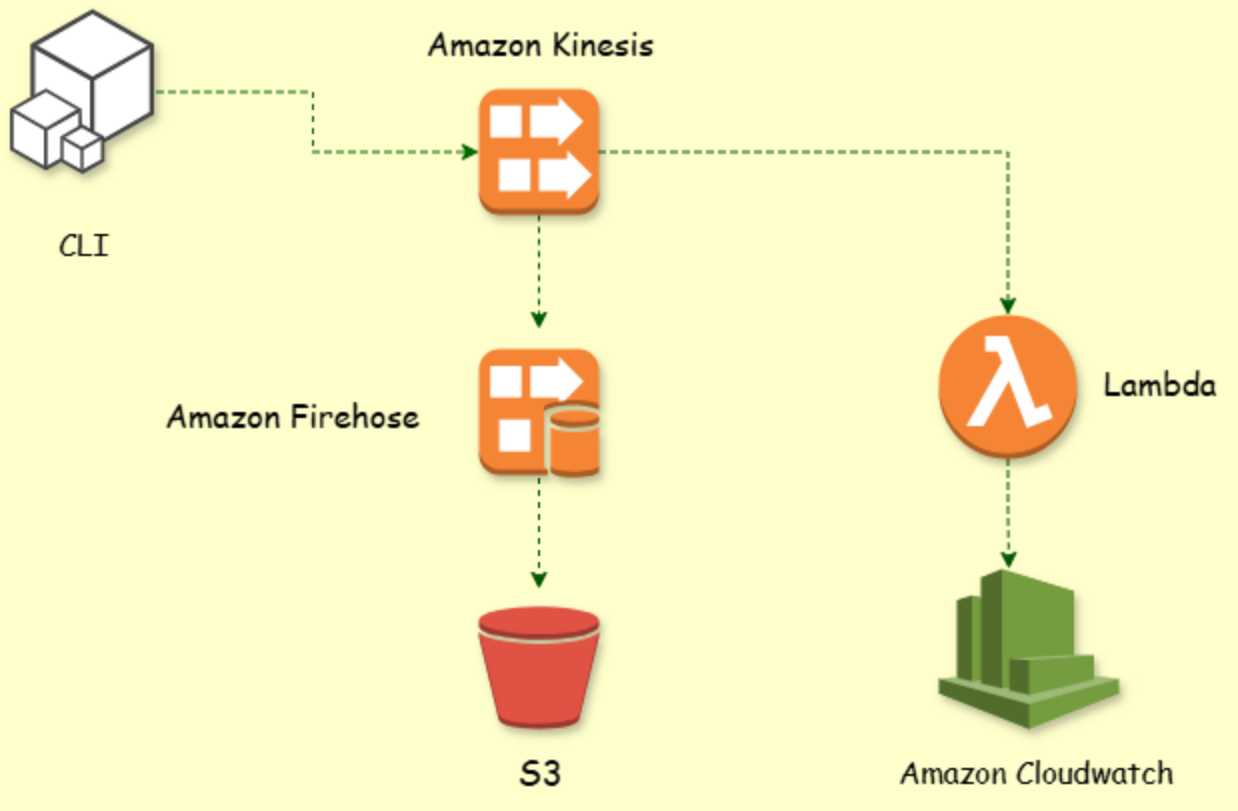
Data Flow Diagram (Textual Representation)

```
┌──────────────────────────────┐
│  Python Producer (IoT Sim)   │
└──────────────┬───────────────┘
               │
               ▼
┌──────────────────────────────┐
│  Amazon Kinesis Data Stream  │
│  (2 Shards)                  │
└──────────────┬───────────────┘
               │
      ┌────────┴────────┐
      │                 │
      ▼                 ▼
┌──────────────────┐  ┌──────────────────────┐
│  AWS Lambda      │  │ Kinesis Data Firehose │
│ - Reads Stream   │  │ - Buffers for 60 sec  │
│ - Filters "ERROR"│  │ - Batch to S3         │
│ - Writes to Logs │  └──────────┬────────────┘
└────────┬─────────┘             │
         │                       ▼
         ▼                 ┌──────────────────┐
┌──────────────────────┐   │  Amazon S3 Bucket │
│ Amazon CloudWatch Logs │  │ (All Sensor Data Storage) │
└──────────────────────┘   └──────────────────┘
```

How to Draw in Lucidchart / draw.io

- **Producer** → Use an icon for a Python script or IoT sensor.
- **Kinesis Data Stream** → AWS Kinesis Stream symbol with **2 shards** labeled.
- **Lambda** → AWS Lambda icon. Connect from stream → Lambda → CloudWatch Logs.
- **Firehose** → AWS Firehose icon. Connect from stream → Firehose → S3 bucket.

- Add arrows to indicate **data flow direction**.



⚡ This setup achieves:

- **Parallelism & scaling** (2 shards).
- **Error handling** (Lambda + CloudWatch).
- **Archival & batch storage** (Firehose → S3).

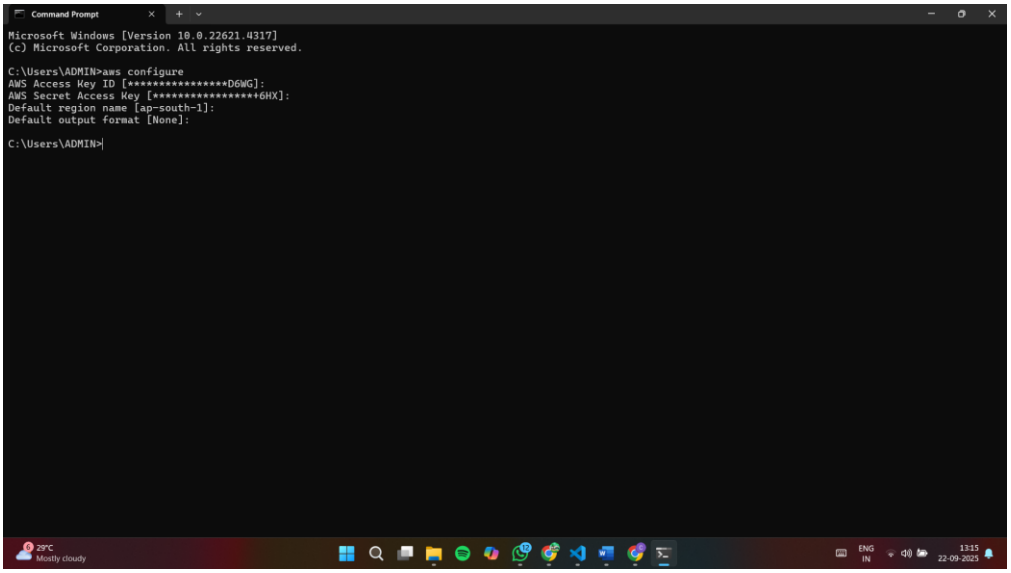
Task 2: Automate Kinesis Stream Creation with AWS CLI/CloudFormation

Part A: Shell Script (AWS CLI)

Steps:

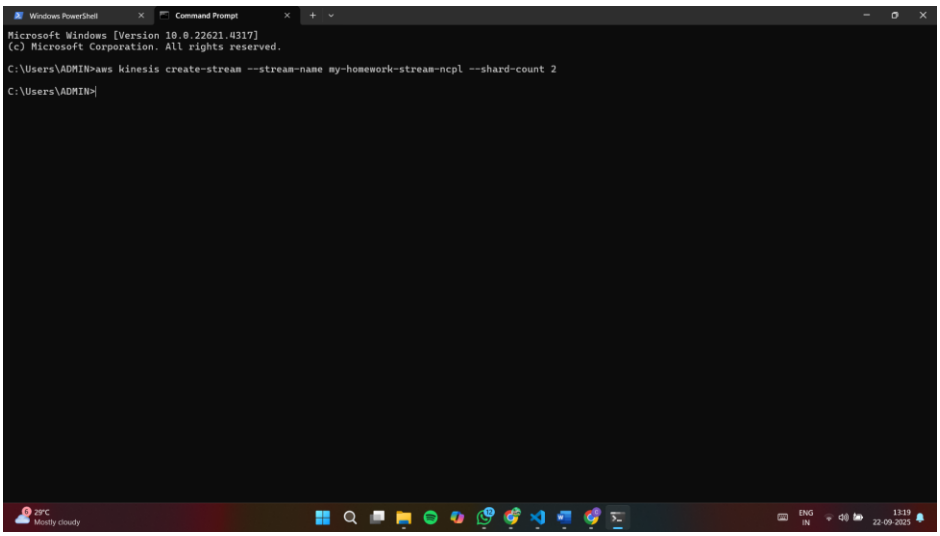
Step 1: Configure aws

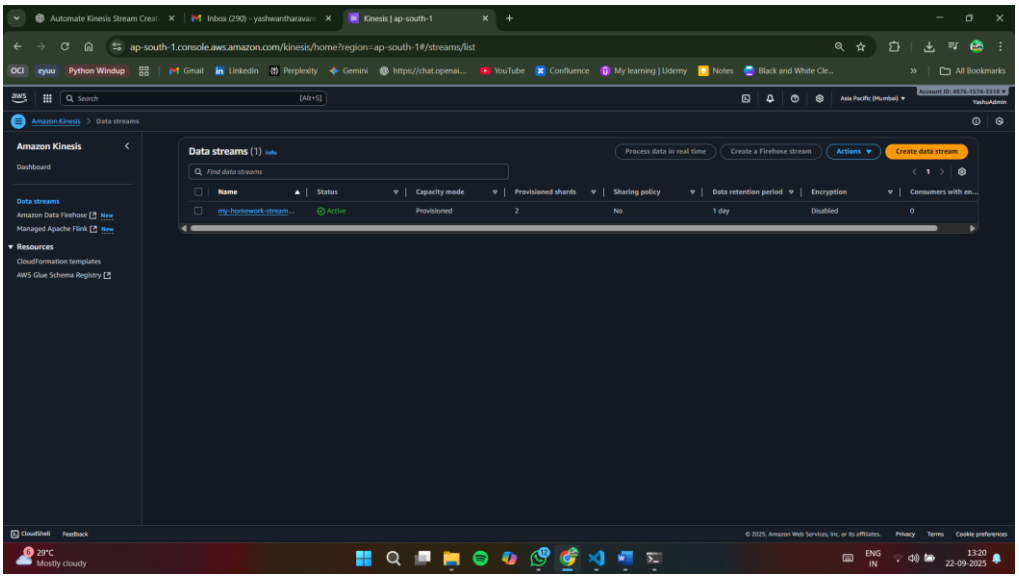
- Use Access Key ID and Secret Access Key to configure CLI



Step 2: Create Kinesis Data Stream

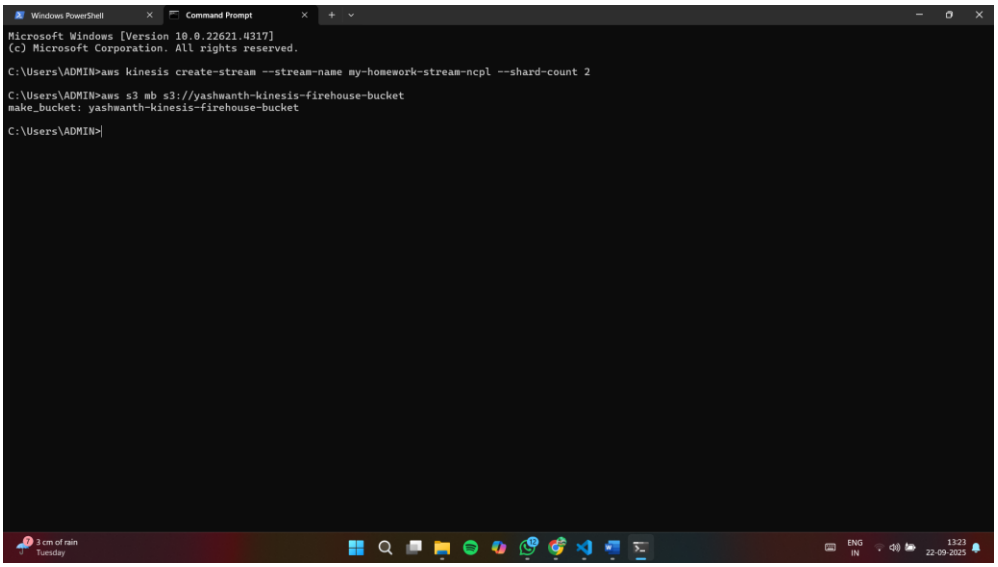
aws kinesis create-stream --stream-name my-homework-stream-ncpl --shard-count 2

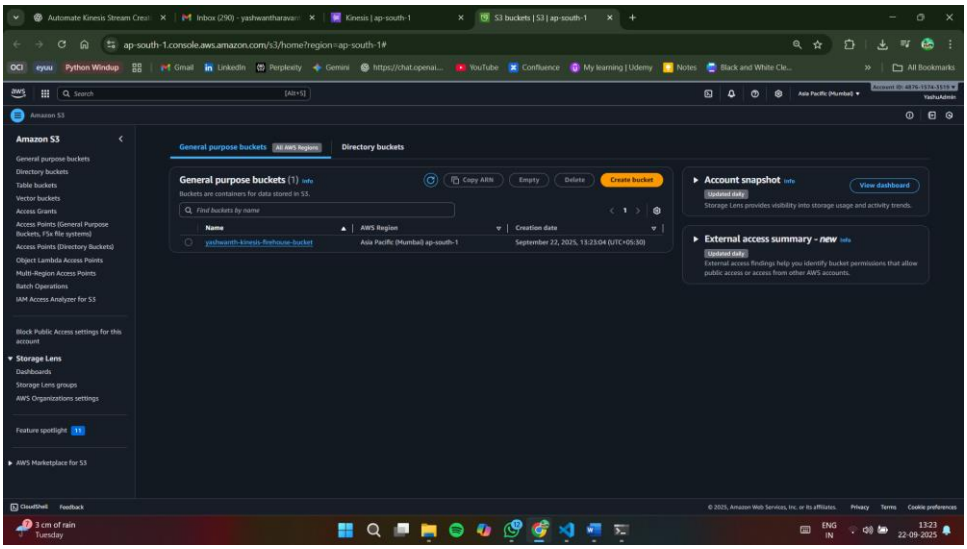




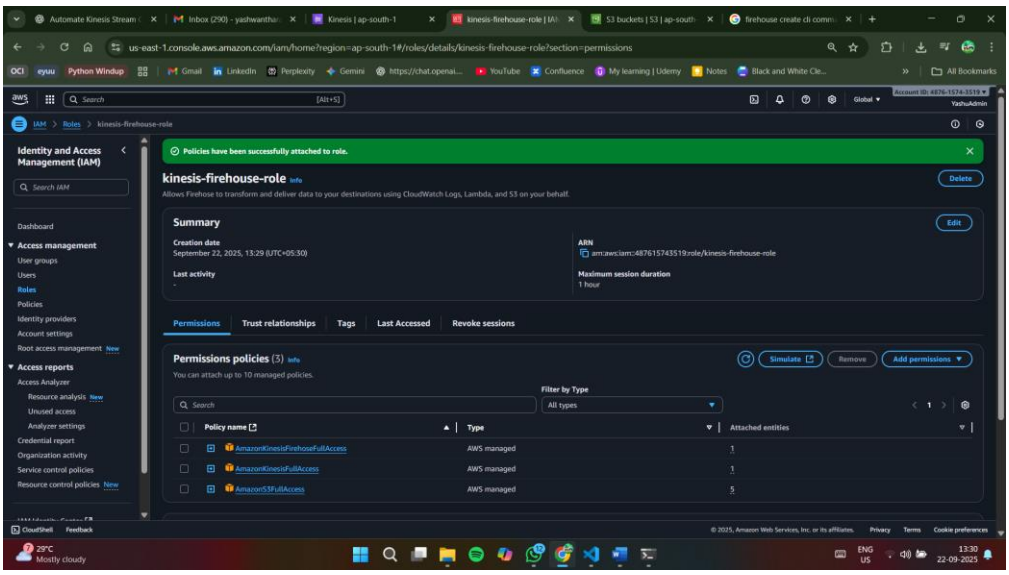
Step 3: Create S3 Bucket

aws s3 mb s3://sumith-kinesis-firehouse-bucket





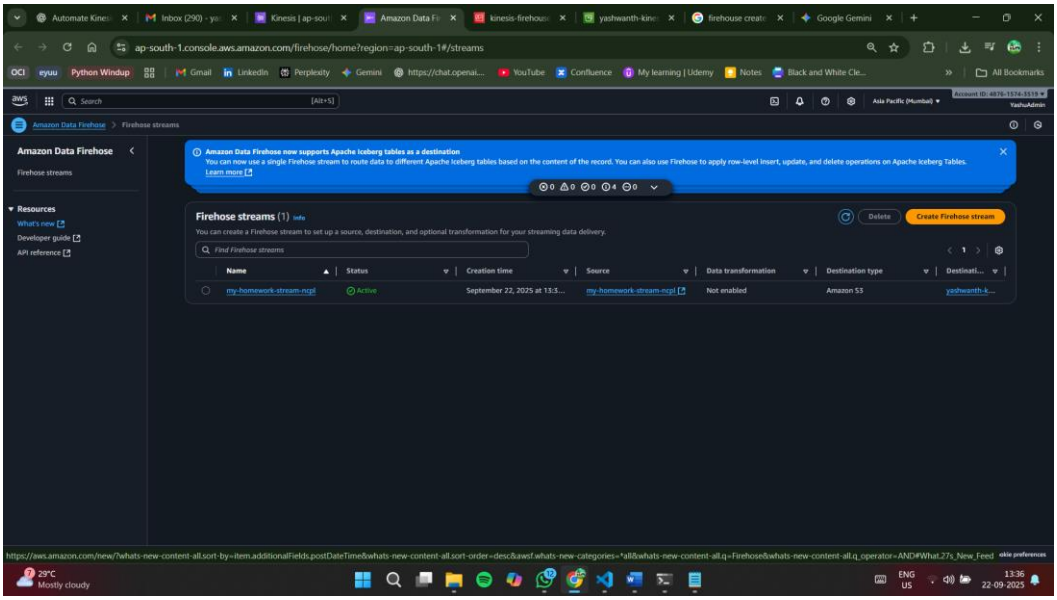
Step 4: Create IAM role

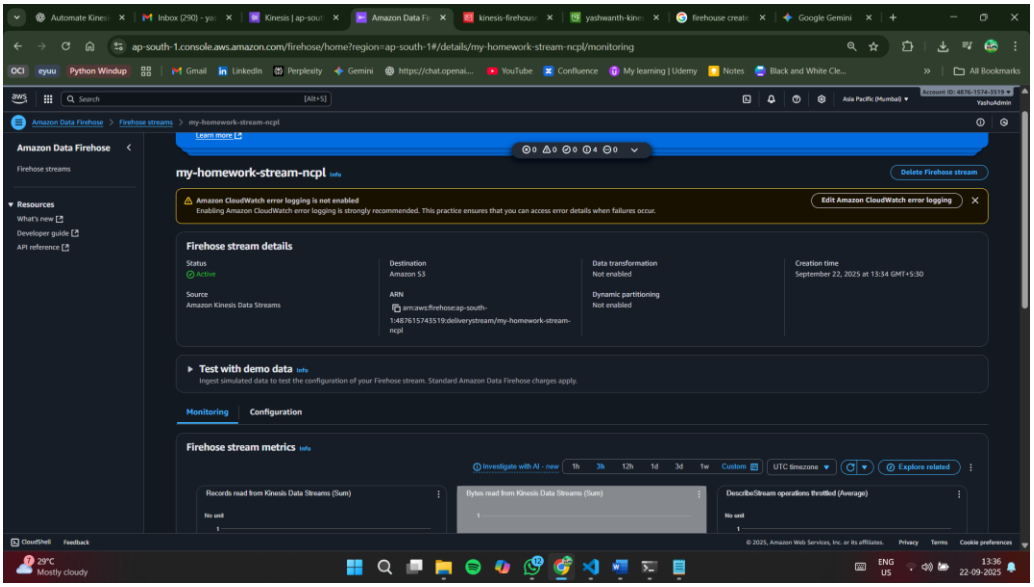


Step 5: Create Firehose Delivery Stream

```
aws firehose create-delivery-stream --delivery-stream-name my-homework-stream-ncpl --delivery-stream-type KinesisStreamAsSource --kinesis-stream-source-configuration RoleARN=arn:aws:iam::487615743519:role/kinesis-firehouse-role,KinesisStreamARN=arn:aws:kinesis:ap-south-1:487615743519:stream/my-homework-stream-ncpl --s3-destination-configuration RoleARN=arn:aws:iam::487615743519:role/kinesis-firehouse-role,BucketARN=arn:aws:s3:::sumith-kinesis-firehouse-bucket
```

```
Command Prompt
Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>aws kinesis create-stream --stream-name my-homework-stream-ncpl --shard-count 2

C:\Users\ADMIN>aws s3 mb s3://yashwanth-kinesis-firehouse-bucket
make_bucket: yashwanth-kinesis-firehouse-bucket

C:\Users\ADMIN>aws firehose create-delivery-stream --delivery-stream-name my-homework-stream-ncpl --delivery-stream-type KinesisStreamAsSource --kinesis-str
eam-source-configuration RoleARN=arn:aws:kinesis:ap-south-1:487615743519:stream/my-homework-stream-ncpl --s3-destination-configuration RoleARN=arn:aws:iam::
487615743519:role/kinesis-firehouse-role,BucketARN=arn:aws:s3:::yashwanth-kinesis-firehouse-bucket

Parameter validation failed:
Missing required parameter in KinesisStreamSourceConfiguration: "KinesisStreamARN"

C:\Users\ADMIN>aws firehose create-delivery-stream --delivery-stream-name my-homework-stream-ncpl --delivery-stream-type KinesisStreamAsSource --kinesis-str
eam-source-configuration RoleARN=arn:aws:iam::487615743519:role/kinesis-firehouse-role,KinesisStreamARN=arn:aws:kinesis:ap-south-1:487615743519:stream/my-ho
mework-stream-ncpl --s3-destination-configuration RoleARN=arn:aws:iam::487615743519:role/kinesis-firehouse-role,BucketARN=arn:aws:s3:::yashwanth-kinesis-fir
ehouse-bucket
{
    "DeliveryStreamARN": "arn:aws:firehose:ap-south-1:487615743519:deliverystream/my-homework-stream-ncpl"
}


C:\Users\ADMIN>
```





Success Criteria

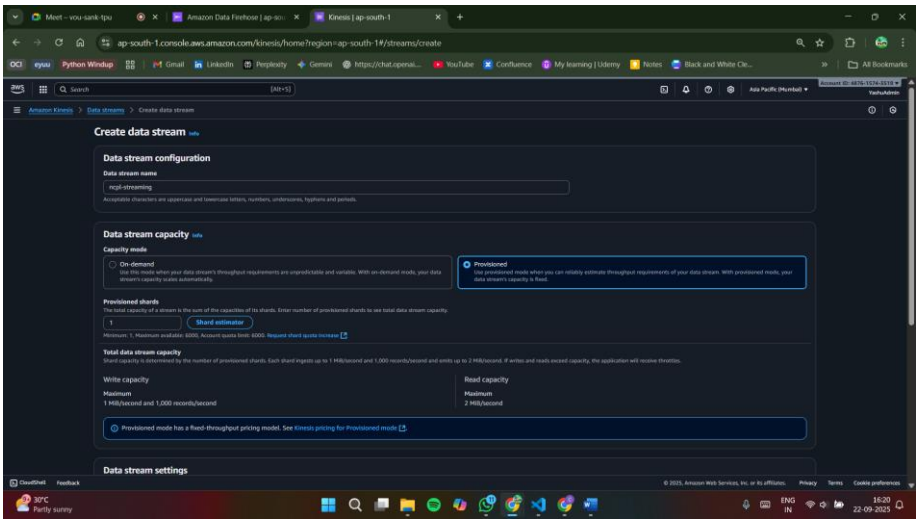
- Run shell script → stream + bucket + firehose created.
- Run aws firehose list-delivery-streams and aws s3 ls to confirm.

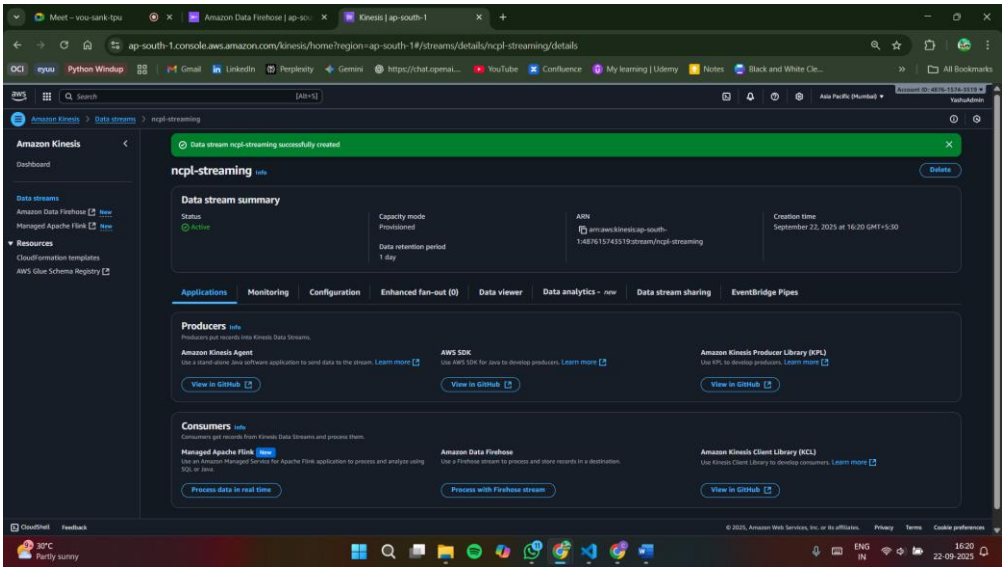
Part B: CloudFormation Template (YAML)

Task 3: Implement and Test Error Handling for Producers

Steps:

Step 1: Create a Kinesis Data Stream





Step 2: Configure CLI

Step 3: Run below commands

CMD:

```
mkdir kinesis-streaming
cd kinesis-streaming
notepad producer.py
```

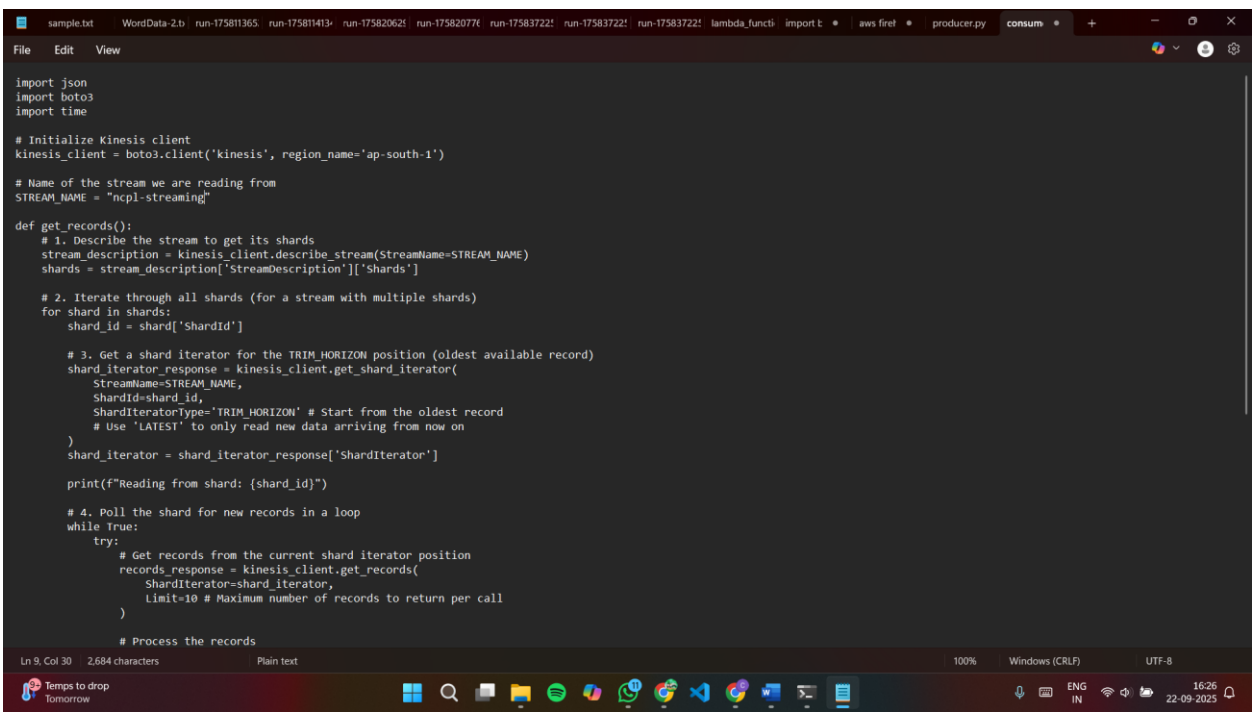
Write the code in producer.py

```
import json
import boto3
import time
import random
from datetime import datetime
from botocore.exceptions import ClientError

# Initialize Kinesis client
kinesis_client = boto3.client("kinesis", region_name="ap-south-1")

# Stream name
STREAM_NAME = "yashwanth"

def generate_status(temperature):
    """Determine sensor status based on temperature thresholds."""
    if temperature < 20:
        return "WARNING"
    elif temperature > 24:
        return "ERROR"
    else:
        return "NORMAL"

def send_record(payload, max_retries=5):
    """Send a record to Kinesis with retry + exponential backoff."""
    retries = 0
    backoff = 0.5  # Initial backoff (seconds)

    while retries < max_retries:
        try:
            response = kinesis_client.put_record(
                StreamName=STREAM_NAME,
                Data=json.dumps(payload),
                PartitionKey=payload["sensor_id"]
            )
            print(f" Record sent. SeqNum: {response['SequenceNumber']}, Data: {payload}")
            return True
        except ClientError as e:
            error_code = e.response['Error']['Code']
            if error_code == "ProvisionedThroughputExceededException":
                print(f" Throttled. Retrying in {backoff:.2f}s...")
                time.sleep(backoff)
                backoff *= 2  # Exponential backoff
                retries += 1
            else:
                print(f" Unexpected error: {e}")
                return False
        except Exception as e:
            print(f" Failed to send record: {e}")
            return False

    print(" Max retries reached. Dropping record.")
    return False
```

```python
def send_data_to_kinesis():
    temperature_values = [19.5, 20.8, 21.6, 22.7, 23.3, 24.5, 25.2, 26.0]

    for i in range(50):  # Send 50 records
        temp = random.choice(temperature_values)
        payload = {
            "sensor_id": f"sensor-{random.randint(1, 5)}",
            "temperature": temp,
            "status": generate_status(temp),
            "timestamp": datetime.utcnow().isoformat()
        }

        send_record(payload)
        time.sleep(0.5)  # Small delay to simulate real-world sensors

if __name__ == "__main__":
    send_data_to_kinesis()
```

CMD:

notepad consumer.py



```python
import json
import boto3
import time

# Initialize Kinesis client
kinesis_client = boto3.client('kinesis', region_name='ap-south-1')

# Name of the stream we are reading from
STREAM_NAME = "ncpl-streaming"

def get_records():
    # 1. Describe the stream to get its shards
    stream_description = kinesis_client.describe_stream(StreamName=STREAM_NAME)
    shards = stream_description['StreamDescription']['Shards']

    # 2. Iterate through all shards (for a stream with multiple shards)
    for shard in shards:
        shard_id = shard['ShardId']

        # 3. Get a shard iterator for the TRIM_HORIZON position (oldest available record)
        shard_iterator_response = kinesis_client.get_shard_iterator(
            StreamName=STREAM_NAME,
            ShardId=shard_id,
            ShardIteratorType='TRIM_HORIZON' # Start from the oldest record
            # Use 'LATEST' to only read new data arriving from now on
        )
        shard_iterator = shard_iterator_response['ShardIterator']

        print(f"Reading from shard: {shard_id}")

        # 4. Poll the shard for new records in a loop
        while True:
            try:
                # Get records from the current shard iterator position
                records_response = kinesis_client.get_records(
                    ShardIterator=shard_iterator,
                    Limit=10 # Maximum number of records to return per call
                )

                # Process the records
```

```python
import json
import boto3
import time

# Initialize Kinesis client
kinesis_client = boto3.client('kinesis', region_name='ap-south-1')

# Name of the stream we are reading from
STREAM_NAME = "ncpl-streaming"

def get_records():
    # 1. Describe the stream to get its shards
    stream_description = kinesis_client.describe_stream(StreamName=STREAM_NAME)
    shards = stream_description['StreamDescription']['Shards']

    # 2. Iterate through all shards (for a stream with multiple shards)
    for shard in shards:
        shard_id = shard['ShardId']

        # 3. Get a shard iterator for the TRIM_HORIZON position (oldest available record)
        shard_iterator_response = kinesis_client.get_shard_iterator(
            StreamName=STREAM_NAME,
            ShardId=shard_id,
            ShardIteratorType='TRIM_HORIZON' # Start from the oldest record
            # Use 'LATEST' to only read new data arriving from now on
        )
        shard_iterator = shard_iterator_response['ShardIterator']

        print(f"Reading from shard: {shard_id}")

        # 4. Poll the shard for new records in a loop
        while True:
            try:
                # Get records from the current shard iterator position
                records_response = kinesis_client.get_records(
                    ShardIterator=shard_iterator,
                    Limit=10 # Maximum number of records to return per call
                )

                # Process the records
                records = records_response['Records']
                if records:
                    print(f"\nReceived {len(records)} records from shard {shard_id}:")
                    for record in records:
                        # The data is base64-encoded bytes, so we need to decode it
                        data_bytes = record['Data']
                        data_str = data_bytes.decode('utf-8')
                        data_json = json.loads(data_str)

                        print(f"PartitionKey: {record['PartitionKey']}, Data: {data_json}")
                else:
                    # No records in this batch
                    print(".", end="", flush=True) # Print a dot to show it's alive

                # Get the next shard iterator for the next call
                shard_iterator = records_response['NextShardIterator']

                # Wait for a second before the next poll to avoid throttling
                time.sleep(1)

            except Exception as e:
                print(f"\nAn error occurred: {e}")
                break
```

```python
if __name__ == "__main__":
    get_records()
```

```
Command Prompt

Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>cd Desktop

C:\Users\ADMIN\Desktop>mkdir kinesis-streaming

C:\Users\ADMIN\Desktop>cd kinesis-streaming

C:\Users\ADMIN\Desktop\kinesis-streaming>notepad producer.py

C:\Users\ADMIN\Desktop\kinesis-streaming>notepad consumer.py

C:\Users\ADMIN\Desktop\kinesis-streaming>ls
'ls' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\ADMIN\Desktop\kinesis-streaming>
```

Step 4: Run the consumer.py file

```
Command Prompt    Command Prompt    Command Prompt - python c

timestamp': '2025-09-22T11:00:35.391696'}
✅ Record sent. SeqNum: 49667324715177312194222629553107104385016818405249908738, Data: {'sensor_id': 'sensor-1', 'temperature': 21.6, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:36.029203'}
✅ Record sent. SeqNum: 49667324715177312194222629553357352029677046713133563906, Data: {'sensor_id': 'sensor-3', 'temperature': 24.5, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:36.586968'}
✅ Record sent. SeqNum: 49667324715177312194222629553550780160815387449806028802, Data: {'sensor_id': 'sensor-2', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:37.284962'}
✅ Record sent. SeqNum: 49667324715177312194222629553897741871044786022946701314, Data: {'sensor_id': 'sensor-1', 'temperature': 19.5, 'status': 'WARNING',
'timestamp': '2025-09-22T11:00:37.840693'}
✅ Record sent. SeqNum: 49667324715177312194222629554105677112018502309715640322, Data: {'sensor_id': 'sensor-3', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:38.414498'}
✅ Record sent. SeqNum: 49667324715177312194222629554355924756678730548879818754, Data: {'sensor_id': 'sensor-4', 'temperature': 22.7, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:38.985658'}
✅ Record sent. SeqNum: 49667324715177312194222629554608590252978188115112886274, Data: {'sensor_id': 'sensor-3', 'temperature': 22.7, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:39.558683'}
✅ Record sent. SeqNum: 49667324715177312194222629554693215060351212226061795330, Data: {'sensor_id': 'sensor-4', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:40.131039'}
✅ Record sent. SeqNum: 49667324715177312194222629554832241529606894581153005570, Data: {'sensor_id': 'sensor-4', 'temperature': 24.5, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:40.722188'}
✅ Record sent. SeqNum: 49667324715177312194222629555037758918941381609572532226, Data: {'sensor_id': 'sensor-2', 'temperature': 26.0, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:41.295477'}
✅ Record sent. SeqNum: 49667324715177312194222629555263828047209317265242587138, Data: {'sensor_id': 'sensor-1', 'temperature': 21.6, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:41.868233'}
✅ Record sent. SeqNum: 49667324715177312194222629555517702469328389460650360834, Data: {'sensor_id': 'sensor-2', 'temperature': 25.2, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:42.512591'}
✅ Record sent. SeqNum: 49667324715177312194222629555864664179557788033791033346, Data: {'sensor_id': 'sensor-5', 'temperature': 24.5, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:43.085530'}
✅ Record sent. SeqNum: 49667324715177312194222629556060510162335358028812910594, Data: {'sensor_id': 'sensor-3', 'temperature': 25.2, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:43.803214'}
✅ Record sent. SeqNum: 49667324715177312194222629556172940263559518610780061698, Data: {'sensor_id': 'sensor-4', 'temperature': 22.7, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:44.379150'}
✅ Record sent. SeqNum: 49667324715177312194222629556257565070932542653009494018, Data: {'sensor_id': 'sensor-3', 'temperature': 19.5, 'status': 'WARNING',
'timestamp': '2025-09-22T11:00:45.061925'}
✅ Record sent. SeqNum: 49667324715177312194222629556414725427482444514440773634, Data: {'sensor_id': 'sensor-1', 'temperature': 24.5, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:45.621166'}
✅ Record sent. SeqNum: 49667324715177312194222629556512648418871229477591973890, Data: {'sensor_id': 'sensor-3', 'temperature': 26.0, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:46.367472'}
✅ Record sent. SeqNum: 49667324715177312194222629556732672918041092056107974658, Data: {'sensor_id': 'sensor-1', 'temperature': 21.6, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:46.940169'}

C:\Users\ADMIN\Desktop\kinesis-streaming>
```

```
Command Prompt    Command Prompt    Command Prompt - python c

imestamp': '2025-09-22T11:00:32.318393'}
✅ Record sent. SeqNum: 49667324715177312194222629552337018637922299414803644418, Data: {'sensor_id': 'sensor-4', 'temperature': 25.2, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:32.893841'}
✅ Record sent. SeqNum: 49667324715177312194222629552413180964558021121529610242, Data: {'sensor_id': 'sensor-2', 'temperature': 22.7, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:33.562040'}
✅ Record sent. SeqNum: 49667324715177312194222629552570341321107922982960889858, Data: {'sensor_id': 'sensor-2', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:34.131921'}
✅ Record sent. SeqNum: 49667324715177312194222629552717830271100907742275043330, Data: {'sensor_id': 'sensor-4', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:34.705437'}
✅ Record sent. SeqNum: 49667324715177312194222629552923347660435394770694569986, Data: {'sensor_id': 'sensor-5', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:35.279997'}
✅ Record sent. SeqNum: 49667324715177312194222629553038195613298784542291656706, Data: {'sensor_id': 'sensor-1', 'temperature': 22.7, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:35.853508'}
✅ Record sent. SeqNum: 49667324715177312194222629553269100444845178783380013058, Data: {'sensor_id': 'sensor-5', 'temperature': 21.6, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:36.427954'}
✅ Record sent. SeqNum: 49667324715177312194222629553425051875575466015636586498, Data: {'sensor_id': 'sensor-4', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:37.045898'}
✅ Record sent. SeqNum: 49667324715177312194222629553733327959577196455186661378, Data: {'sensor_id': 'sensor-4', 'temperature': 21.6, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:37.617853'}
✅ Record sent. SeqNum: 49667324715177312194222629554041604043578926963456212994, Data: {'sensor_id': 'sensor-3', 'temperature': 24.5, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:38.191498'}
✅ Record sent. SeqNum: 49667324715177312194222629554214480435783818935439196162, Data: {'sensor_id': 'sensor-2', 'temperature': 22.7, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:38.779834'}
✅ Record sent. SeqNum: 49667324715177312194222629554543308258718998139678752770, Data: {'sensor_id': 'sensor-1', 'temperature': 24.5, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:39.352111'}
✅ Record sent. SeqNum: 49667324715177312194222629554672663321417763461372313602, Data: {'sensor_id': 'sensor-1', 'temperature': 26.0, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:39.923276'}
✅ Record sent. SeqNum: 49667324715177312194222629554817734419771519031056531458, Data: {'sensor_id': 'sensor-3', 'temperature': 22.7, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:40.498773'}
✅ Record sent. SeqNum: 49667324715177312194222629555035341067302152282503643138, Data: {'sensor_id': 'sensor-5', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:41.248094'}
✅ Record sent. SeqNum: 49667324715177312194222629555288006563601609848736710658, Data: {'sensor_id': 'sensor-2', 'temperature': 22.7, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:41.820187'}
✅ Record sent. SeqNum: 49667324715177312194222629555518911395148004089825067010, Data: {'sensor_id': 'sensor-3', 'temperature': 24.5, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:00:42.512591'}
✅ Record sent. SeqNum: 49667324715177312194222629555865873105377402731685216258, Data: {'sensor_id': 'sensor-1', 'temperature': 21.6, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:43.085530'}
✅ Record sent. SeqNum: 49667324715177312194222629556061719088154972657987616770, Data: {'sensor_id': 'sensor-1', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:00:43.803214'}

C:\Users\ADMIN\Desktop\kinesis-streaming>

30°C Partly sunny    ENG IN    16:33 22-09-2025
```

```
Command Prompt    Command Prompt    Command Prompt - python c

Received 4 records from shard shardId-000000000000:
PartitionKey: sensor-2, Data: {'sensor_id': 'sensor-2', 'temperature': 22.7, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:38.779834'}
PartitionKey: sensor-4, Data: {'sensor_id': 'sensor-4', 'temperature': 22.7, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:38.985658'}
PartitionKey: sensor-1, Data: {'sensor_id': 'sensor-1', 'temperature': 24.5, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:39.352111'}
PartitionKey: sensor-3, Data: {'sensor_id': 'sensor-3', 'temperature': 22.7, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:39.558683'}

Received 4 records from shard shardId-000000000000:
PartitionKey: sensor-1, Data: {'sensor_id': 'sensor-1', 'temperature': 26.0, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:39.923276'}
PartitionKey: sensor-4, Data: {'sensor_id': 'sensor-4', 'temperature': 23.3, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:40.131039'}
PartitionKey: sensor-3, Data: {'sensor_id': 'sensor-3', 'temperature': 22.7, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:40.498773'}
PartitionKey: sensor-4, Data: {'sensor_id': 'sensor-4', 'temperature': 24.5, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:40.722188'}

Received 2 records from shard shardId-000000000000:
PartitionKey: sensor-5, Data: {'sensor_id': 'sensor-5', 'temperature': 23.3, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:41.248094'}
PartitionKey: sensor-2, Data: {'sensor_id': 'sensor-2', 'temperature': 26.0, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:41.295477'}

Received 5 records from shard shardId-000000000000:
PartitionKey: sensor-1, Data: {'sensor_id': 'sensor-1', 'temperature': 21.6, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:41.868233'}
PartitionKey: sensor-2, Data: {'sensor_id': 'sensor-2', 'temperature': 22.7, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:41.820187'}
PartitionKey: sensor-2, Data: {'sensor_id': 'sensor-2', 'temperature': 25.2, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:42.512591'}
PartitionKey: sensor-3, Data: {'sensor_id': 'sensor-3', 'temperature': 24.5, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:42.512591'}
PartitionKey: sensor-5, Data: {'sensor_id': 'sensor-5', 'temperature': 24.5, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:43.085530'}

Received 3 records from shard shardId-000000000000:
PartitionKey: sensor-1, Data: {'sensor_id': 'sensor-1', 'temperature': 21.6, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:43.085530'}
PartitionKey: sensor-3, Data: {'sensor_id': 'sensor-3', 'temperature': 25.2, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:43.803214'}
PartitionKey: sensor-1, Data: {'sensor_id': 'sensor-1', 'temperature': 23.3, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:43.803214'}

Received 2 records from shard shardId-000000000000:
PartitionKey: sensor-4, Data: {'sensor_id': 'sensor-4', 'temperature': 22.7, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:44.379150'}
PartitionKey: sensor-3, Data: {'sensor_id': 'sensor-3', 'temperature': 19.5, 'status': 'WARNING', 'timestamp': '2025-09-22T11:00:45.061925'}

Received 2 records from shard shardId-000000000000:
PartitionKey: sensor-1, Data: {'sensor_id': 'sensor-1', 'temperature': 24.5, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:45.621166'}
PartitionKey: sensor-3, Data: {'sensor_id': 'sensor-3', 'temperature': 26.0, 'status': 'ERROR', 'timestamp': '2025-09-22T11:00:46.367472'}

Received 1 records from shard shardId-000000000000:
PartitionKey: sensor-1, Data: {'sensor_id': 'sensor-1', 'temperature': 21.6, 'status': 'NORMAL', 'timestamp': '2025-09-22T11:00:46.940169'}
.....................................................................................................................................
.................

30°C Partly sunny    ENG IN    16:34 22-09-2025
```

What Changed:

1. Retry with exponential backoff
    - Handles ProvisionedThroughputExceededException.
    - Retries up to max_retries (default 5).
    - Backoff doubles on each retry (0.5s → 1s → 2s → 4s ...).
2. Structured Data
   Each record includes:
    - sensor_id
    - temperature
    - status ("NORMAL", "WARNING", "ERROR")
    - timestamp
3. Scalability Testing
    - You can run multiple instances of producer.py in separate terminals:
    - python producer.py &
    - python producer.py &
    - This will increase load and may trigger throttling → your retry logic should handle it.

Your consumer.py already works fine — you'll now see the status field in output along with sensor_id and timestamp.

Task 4: Consumer Application with Record Processing Logic

Prerequisites

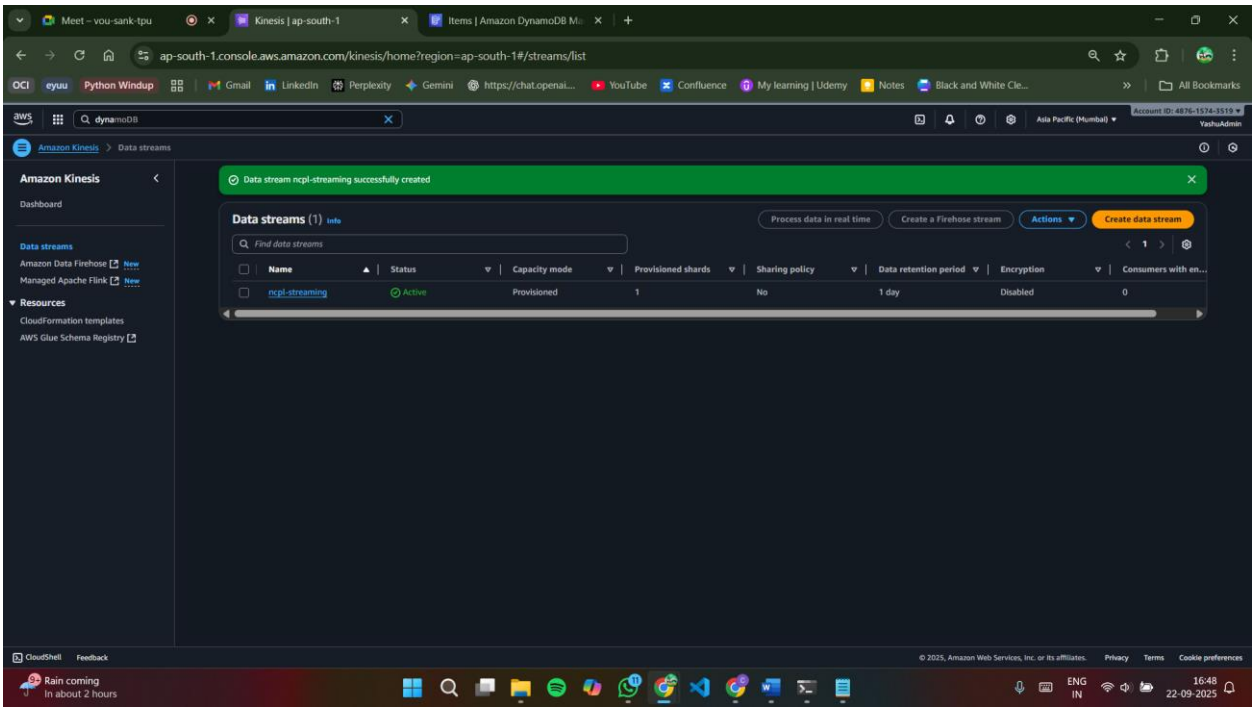
1. AWS account with permissions to manage Kinesis, DynamoDB, IAM, CloudWatch.
2. AWS CLI configured (aws configure) or environment variables set for AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN (if used).
3. Python 3.8+ installed.
4. boto3 and botocore installed in a virtualenv.

Steps:

Step 1: Create Kinesis Stream

Choose a stream name and shard count depending on throughput. Example CLI:

aws kinesis create-stream --stream-name sumith--shard-count 1 --region ap-south-1



Step 2: Create DynamoDB table for checkpoints

The consumer stores the last processed sequence_number per shard in a DynamoDB table.

Example CLI:

```
aws dynamodb create-table \
--table-name KinesisCheckpoints \
--attribute-definitions AttributeName=shard_id,AttributeType=S \
--key-schema AttributeName=shard_id,KeyType=HASH \
--billing-mode PAY_PER_REQUEST \
--region ap-south-1
```

The application can also create this table programmatically if it doesn't exist.

Step 3: IAM Policy (permissions)

Minimal permissions needed for producers and consumers. Put this on the user/role running the code.

```
{
"Version": "2012-10-17",
"Statement": [
{
"Effect": "Allow",
"Action": [
"kinesis:PutRecord",
"kinesis:PutRecords",
"kinesis:DescribeStream",
"kinesis:GetShardIterator",
"kinesis:GetRecords"
],
"Resource": "arn:aws:kinesis:ap-south-1:123456789012:stream/yashwanth"
},
{
"Effect": "Allow",
"Action": [
"dynamodb:GetItem",
"dynamodb:PutItem",
"dynamodb:UpdateItem",
"dynamodb:CreateTable",
"dynamodb:DescribeTable"
],
"Resource": "*"
},
{
"Effect": "Allow",
"Action": ["cloudwatch:PutMetricData"],
"Resource": "*"
}
]
}
```

Note: Narrow down dynamodb resource ARNs if you want strong least-privilege.

Step 4: Create Producer

If not use previous tasks producer

- Generates a payload with sensor_id, temperature, status, timestamp.
- Calls put_record.
- Handles ProvisionedThroughputExceededException with exponential backoff retries.

How to run:

python producer.py

Step 5: Create Consumer.py (Consumer with Logic & Checkpointing)

- Reads records from each shard in a loop using get_shard_iterator and get_records.
- Parses record JSON and applies business logic: increments an ERROR counter if status == 'ERROR'.
- Prints a summary of total ERRORs every 30 seconds.
- Writes the last processed sequence_number for each shard to the DynamoDB checkpoints table.
- On restart, it reads the checkpoint and resumes from AFTER_SEQUENCE_NUMBER (so it doesn't reprocess the same record).

CMD:

notepad consumer.py

Code:

```
import json
import boto3
import time
from datetime import datetime

# AWS clients
kinesis_client = boto3.client("kinesis", region_name="ap-south-1")
dynamodb = boto3.resource("dynamodb", region_name="ap-south-1")

# Stream + DynamoDB table name
STREAM_NAME = "ncpl-streaming"
CHECKPOINT_TABLE = "KinesisCheckpoints"

# Create DynamoDB table if not exists (hash key: shard_id)
def ensure_checkpoint_table():
    try:
        table = dynamodb.create_table(
            TableName=CHECKPOINT_TABLE,
            KeySchema=[{"AttributeName": "shard_id", "KeyType": "HASH"}],
            AttributeDefinitions=[{"AttributeName": "shard_id", "AttributeType": "S"}],
            BillingMode="PAY_PER_REQUEST"
        )
        table.wait_until_exists()
        print(f"✅ DynamoDB table {CHECKPOINT_TABLE} created")
    except dynamodb.meta.client.exceptions.ResourceInUseException:
        # Table already exists
        pass
    return dynamodb.Table(CHECKPOINT_TABLE)

checkpoint_table = ensure_checkpoint_table()
```

```python
def get_checkpoint(shard_id):
    """Retrieve last processed sequence number for shard."""
    response = checkpoint_table.get_item(Key={"shard_id": shard_id})
    return response.get("Item", {}).get("sequence_number")

def update_checkpoint(shard_id, seq_number):
    """Update checkpoint in DynamoDB."""
    checkpoint_table.put_item(Item={
        "shard_id": shard_id,
        "sequence_number": seq_number
    })

def process_stream():
    # Track error counts
    error_count = 0
    last_summary_time = time.time()

    # Describe stream shards
    stream_description = kinesis_client.describe_stream(StreamName=STREAM_NAME)
    shards = stream_description["StreamDescription"]["Shards"]

    for shard in shards:
        shard_id = shard["ShardId"]
        checkpoint = get_checkpoint(shard_id)

        # Choose iterator type based on checkpoint
        if checkpoint:
            iterator_type = "AFTER_SEQUENCE_NUMBER"
            shard_iterator_response = kinesis_client.get_shard_iterator(
                StreamName=STREAM_NAME,
                ShardId=shard_id,
                ShardIteratorType=iterator_type,
                StartingSequenceNumber=checkpoint
            )
        else:
            iterator_type = "TRIM_HORIZON"
            shard_iterator_response = kinesis_client.get_shard_iterator(
                StreamName=STREAM_NAME,
                ShardId=shard_id,
                ShardIteratorType=iterator_type
            )

        shard_iterator = shard_iterator_response["ShardIterator"]
        print(f"🔑 Reading from shard {shard_id} (starting at {iterator_type})")

        # Continuously poll for new records
        while True:
            try:
                response = kinesis_client.get_records(
                    ShardIterator=shard_iterator,
                    Limit=100
                )

                records = response["Records"]
                if records:
                    for record in records:
                        data = json.loads(record["Data"].decode("utf-8"))
                        seq_number = record["SequenceNumber"]

                        # Apply business logic
                        if data.get("status") == "ERROR":
                            error_count += 1
```

```
                        # Update checkpoint
                        update_checkpoint(shard_id, seq_number)

                # Print error summary every 30s
                now = time.time()
                if now - last_summary_time >= 30:
                    print(f"[{datetime.utcnow().isoformat()}] ⚠ Total ERROR records so far:
{error_count}")
                    last_summary_time = now

                # Move iterator forward
                shard_iterator = response["NextShardIterator"]
                time.sleep(1)

            except Exception as e:
                print(f"❌ Error reading from shard {shard_id}: {e}")
                break

if __name__ == "__main__":
    process_stream()
```

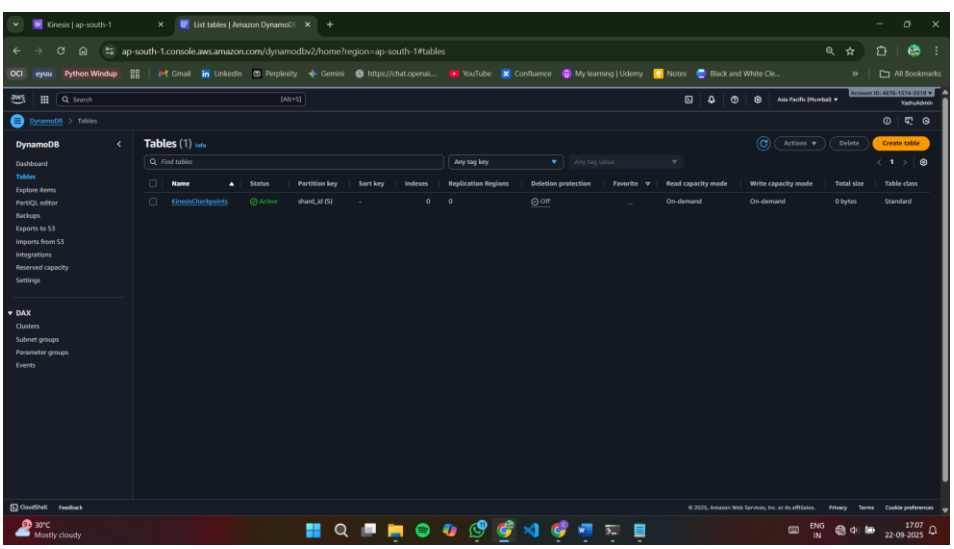
How to run:

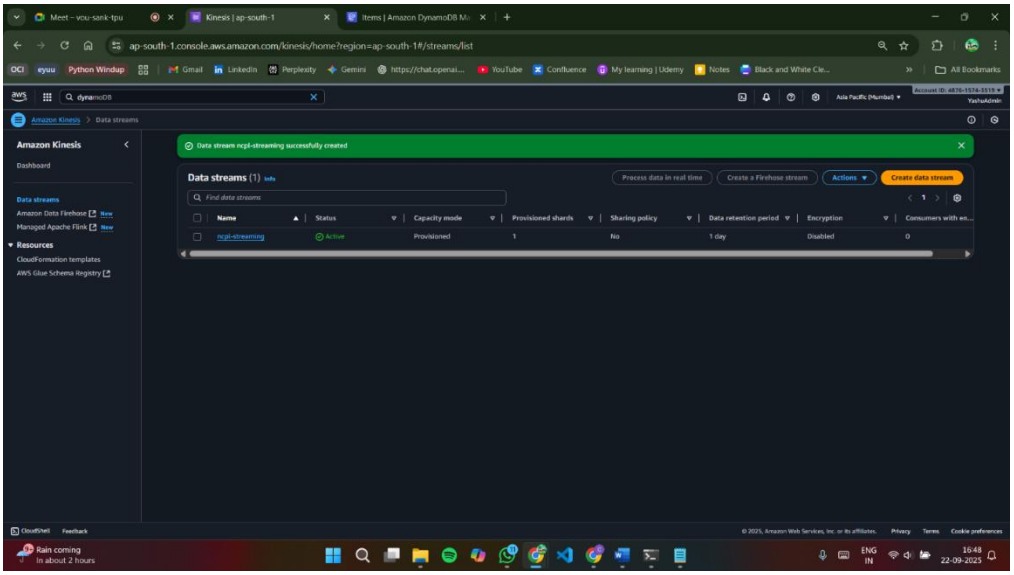
python consumer.py

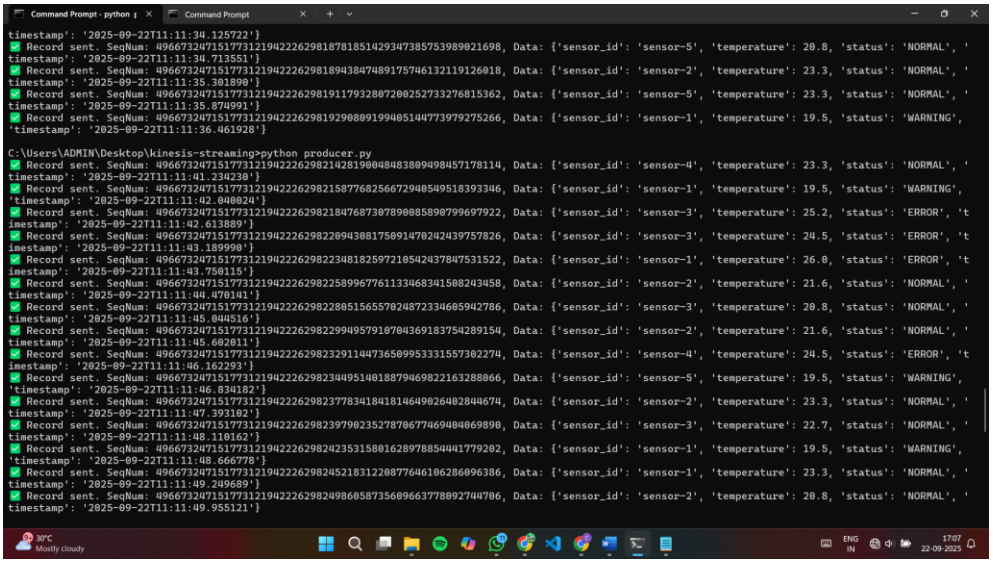
Step 6: Testing Plan

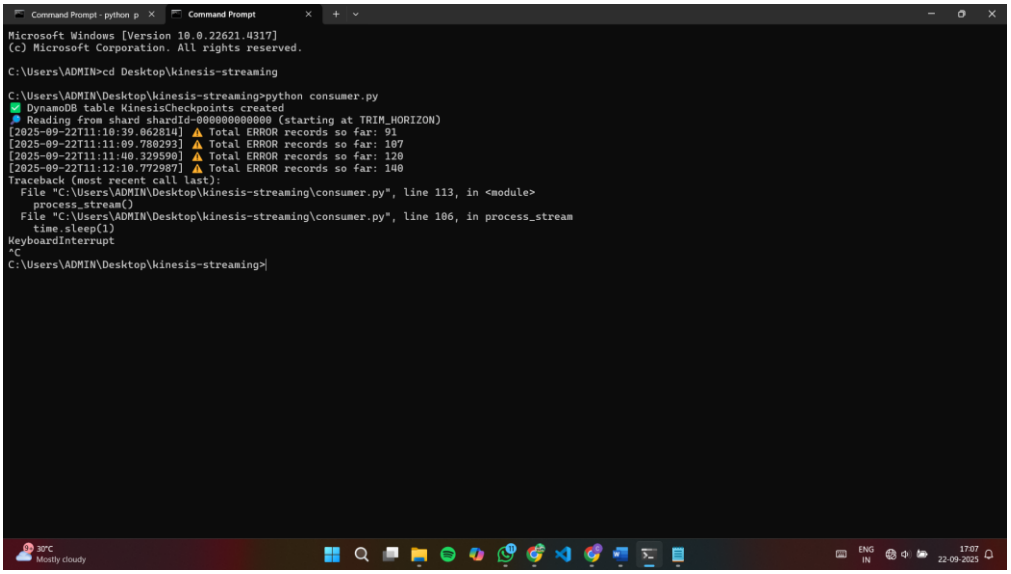
1. Start a consumer: python consumer_with_logic.py.
2. Start a single producer: python producer.py.
3. Expect to see consumer print data and ERROR summary every 30 seconds.

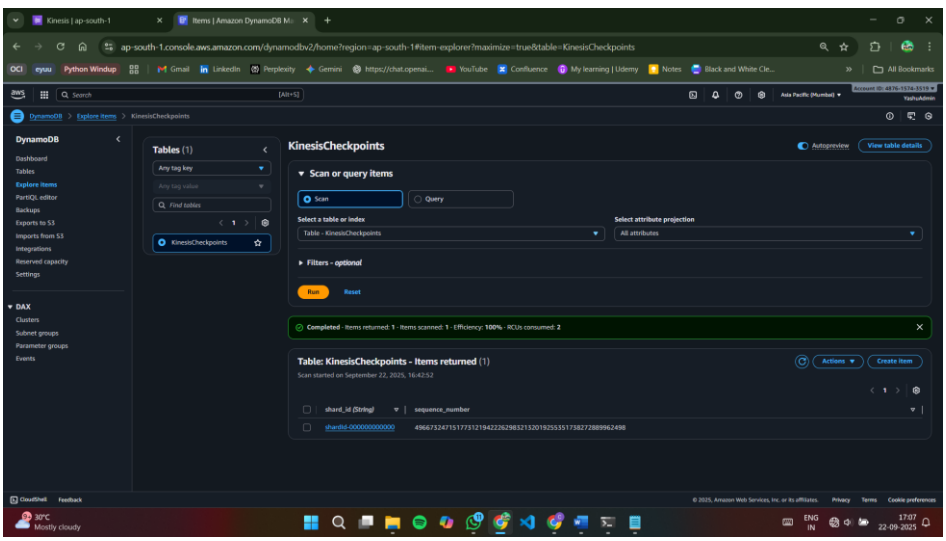
Expected: All records are processed; checkpoint table populated with a sequence number for each shard.




Kinesis | ap-south-1
ap-south-1.console.aws.amazon.com/kinesis/home?region=ap-south-1#/streams/list
Amazon Kinesis
Data stream ncpl-streaming successfully created
Data streams (1)
Process data in real time
Create a Firehose stream
Actions
Create data stream
Name
Status
Capacity mode
Provisioned shards
Sharing policy
Data retention period
Encryption
ncpl-streaming
Active
Provisioned
1
No
1 day
Disabled
0


Command Prompt - python
Command Prompt
timestamp': '2025-09-22T11:11:34.125722'}
Record sent. SeqNum: 49667324715177331219422262981878185142934738575398902169 8, Data: {'sensor_id': 'sensor-5', 'temperature': 20.8, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:34.713551'}
Record sent. SeqNum: 49667324715177331219422262981894384748917574613211912601 8, Data: {'sensor_id': 'sensor-2', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:35.301890'}
Record sent. SeqNum: 49667324715177331219422262981911793280720025273327681536 2, Data: {'sensor_id': 'sensor-5', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:35.874991'}
Record sent. SeqNum: 49667324715177331219422262981929080919940514477397927526 6, Data: {'sensor_id': 'sensor-1', 'temperature': 19.5, 'status': 'WARNING',
'timestamp': '2025-09-22T11:11:36.461928'}
C:\Users\ADMIN\Desktop\kinesis-streaming>python producer.py
Record sent. SeqNum: 49667324715177331219422262982142819004848380940845717811 4, Data: {'sensor_id': 'sensor-4', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:41.234230'}
Record sent. SeqNum: 49667324715177331219422262982158776825667294054951839334 6, Data: {'sensor_id': 'sensor-1', 'temperature': 19.5, 'status': 'WARNING',
'timestamp': '2025-09-22T11:11:42.040024'}
Record sent. SeqNum: 49667324715177331219422262982184768730789008589079969792 2, Data: {'sensor_id': 'sensor-3', 'temperature': 25.2, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:11:42.613889'}
Record sent. SeqNum: 49667324715177331219422262982209430817509147024243975782 6, Data: {'sensor_id': 'sensor-3', 'temperature': 24.5, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:11:43.189990'}
Record sent. SeqNum: 49667324715177331219422262982234818259721054243784753152 2, Data: {'sensor_id': 'sensor-1', 'temperature': 26.0, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:11:43.750115'}
Record sent. SeqNum: 49667324715177331219422262982258996776113346834150824345 8, Data: {'sensor_id': 'sensor-2', 'temperature': 21.6, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:44.470141'}
Record sent. SeqNum: 49667324715177331219422262982280515655702487233460594278 6, Data: {'sensor_id': 'sensor-3', 'temperature': 20.8, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:45.044516'}
Record sent. SeqNum: 49667324715177331219422262982299495791070436918375428915 4, Data: {'sensor_id': 'sensor-2', 'temperature': 21.6, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:45.602011'}
Record sent. SeqNum: 49667324715177331219422262982329114473650995333155730227 4, Data: {'sensor_id': 'sensor-4', 'temperature': 24.5, 'status': 'ERROR', 't
imestamp': '2025-09-22T11:11:46.162293'}
Record sent. SeqNum: 49667324715177331219422262982344951401887946982216328806 6, Data: {'sensor_id': 'sensor-5', 'temperature': 19.5, 'status': 'WARNING',
'timestamp': '2025-09-22T11:11:46.834182'}
Record sent. SeqNum: 49667324715177331219422262982377834184181464902640284467 4, Data: {'sensor_id': 'sensor-2', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:47.393102'}
Record sent. SeqNum: 49667324715177331219422262982397902352787067746940406989 0, Data: {'sensor_id': 'sensor-3', 'temperature': 22.7, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:48.110162'}
Record sent. SeqNum: 49667324715177331219422262982423531580162897885444177920 2, Data: {'sensor_id': 'sensor-1', 'temperature': 19.5, 'status': 'WARNING',
'timestamp': '2025-09-22T11:11:48.666778'}
Record sent. SeqNum: 49667324715177331219422262982452183122087764610628609638 6, Data: {'sensor_id': 'sensor-1', 'temperature': 23.3, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:49.249689'}
Record sent. SeqNum: 49667324715177331219422262982498605873560966377880927447 06, Data: {'sensor_id': 'sensor-2', 'temperature': 20.8, 'status': 'NORMAL', '
timestamp': '2025-09-22T11:11:49.955121'}





Producer Applications

We started with producers that generate and push fake IoT sensor data into a Kinesis Data Stream:

- Each record contains:

{

"sensor_id": "sensor-5",

"temperature": 29.7,

"status": "OK",

"timestamp": "2025-09-22T16:45:12.000Z"

}

- We made sure producers retry with exponential backoff when throttled.

- Later, we improved them by using batch writes (put_records) so one API call could send hundreds of records → much more efficient for high throughput.

Summary

This consumer reads from each shard in a Kinesis stream, parses JSON data, and applies business logic to count "ERROR" events. Every 30 seconds, it prints a running summary of error counts. It also implements **checkpointing** by saving the last processed sequence number per shard, so if restarted, it resumes from the last checkpoint instead of reprocessing all records. This ensures reliable, fault-tolerant consumption.

Step 1: Estimate Monthly Cost in AWS Pricing Calculator

1. Go to AWS Pricing Calculator.
2. Click **Create estimate** → search for **Kinesis Data Streams**.
   - Select **2 shards**.
   - Running 24/7 → 2 shards × 730 hours = 1,460 shard-hours.
   - Cost: 1,460 × $0.015 = **~$21.90/month**.
   - Add **5 GB/day ingestion** = ~150 GB/month. At $0.014 per million PUT payload units (~1KB records), ~**$2.20/month**.
   - **Subtotal: ~$24.10/month**.
3. Add **Kinesis Data Firehose**:
   - Data transformation enabled.
   - 150 GB/month × ($0.029 ingestion + $0.018 transformation) = **$7.05/month**.
4. Add **Amazon S3**:
   - Store 150 GB/month in S3 Standard at $0.023/GB.
   - 150 × 0.023 = **$3.45/month**.
5. **Total monthly estimate** = ~$34.60/month.

In the calculator you'll see a neat breakdown of each service. Save/export PDF for your documentation.

Step 2: Create a Billing Alarm in AWS Budgets

1. Open the **AWS Billing Console** → **Budgets**.
2. Click **Create budget** → choose **Cost budget**.
3. Set details:
   - **Period**: Monthly.
   - **Budgeted amount**: $10.
4. Set notifications:
   - Add an **alert when forecasted cost > 100%**.
   - Add an **email recipient**. (AWS will send a verification email before alerts start).
5. Review and click **Create budget**.

Now, whenever projected costs exceed $10, you'll get notified.

Step 3: Create a CloudWatch Dashboard for Kinesis Metrics

Option A: Console (easy way)

1. Open **CloudWatch Console → Dashboards → Create dashboard**.
2. Name it (e.g., Kinesis-Monitoring).
3. Add **line graph widgets** for each metric:
    - Namespace: AWS/Kinesis.
    - StreamName: *your stream*.
    - Metrics:
        - IncomingBytes
        - OutgoingBytes
        - ReadProvisionedThroughputExceeded
        - WriteProvisionedThroughputExceeded
4. Arrange the graphs and save.

Option B: JSON Import (fast)

Paste this JSON (replace STREAM_NAME with your actual stream name, region as needed):

```
{
  "widgets": [
    {
      "type": "metric",
      "x": 0, "y": 0, "width": 12, "height": 6,
      "properties": {
        "metrics": [
          [ "AWS/Kinesis", "IncomingBytes", "StreamName", "STREAM_NAME" ]
        ],
        "title": "Incoming Bytes",
        "region": "us-east-1",
        "stat": "Sum",
        "period": 300
      }
    },
    {
      "type": "metric",
      "x": 12, "y": 0, "width": 12, "height": 6,
      "properties": {
        "metrics": [
          [ "AWS/Kinesis", "OutgoingBytes", "StreamName", "STREAM_NAME" ]
        ],
        "title": "Outgoing Bytes",
        "region": "us-east-1",
        "stat": "Sum",
        "period": 300
```

```
      }
    },
    {
      "type": "metric",
      "x": 0, "y": 6, "width": 12, "height": 6,
      "properties": {
        "metrics": [
          [ "AWS/Kinesis", "ReadProvisionedThroughputExceeded", "StreamName", "STREAM_NAME" ]
        ],
        "title": "Read Throttle Events",
        "region": "us-east-1",
        "stat": "Sum",
        "period": 300
      }
    },
    {
      "type": "metric",
      "x": 12, "y": 6, "width": 12, "height": 6,
      "properties": {
        "metrics": [
          [ "AWS/Kinesis", "WriteProvisionedThroughputExceeded", "StreamName", "STREAM_NAME" ]
        ],
        "title": "Write Throttle Events",
        "region": "us-east-1",
        "stat": "Sum",
        "period": 300
      }
    }
  ]
}
```